Introduction

Every so often a LeetCode problem shows up wearing a disguise. It reads like it wants heavy math — factorization, digit DP, some clever number-theory identity. LeetCode 3345, Smallest Divisible Digit Product I, is one of those problems. The moment you actually sit with it, the disguise falls off, and what's left is a simple linear scan with one elegant observation that keeps it fast.

This problem is worth learning for a specific reason: it tests whether you can recognize when brute force is actually fine, not because you're being lazy, but because you can prove a bound on how far it has to run. That's a real interview skill. Interviewers love asking "isn't this just brute force?" and watching whether you can answer with a number instead of a shrug. By the end of this article, you'll be able to answer that question for this problem in one sentence.

This is also a great problem for practicing digit manipulation in Python — converting numbers to strings, walking through characters, and using the modulo operator correctly. Those are small mechanical skills, but they show up constantly across the "digit problems" family on LeetCode.

Problem Overview

You're given two integers, `n` and `t`. You need to find the smallest integer that is greater than or equal to `n` such that when you multiply all of its digits together, the resulting product is divisible by `t`.

In other words: start scanning from `n` upward. For each candidate number, multiply its digits into a single product. The first candidate whose digit product is evenly divisible by `t` (remainder zero) is your answer.

There's no requirement to skip `n` itself — if `n` already satisfies the condition, `n` is the answer.

Example

Let's say `n = 15` and `t = 3`.

- Check 15: digits are `1` and `5`. Product = `1 × 5 = 5`. Is `5` divisible by `3`? No (`5 % 3 = 2`).
- Check 16: digits are `1` and `6`. Product = `1 × 6 = 6`. Is `6` divisible by `3`? Yes (`6 % 3 = 0`).

Answer: **16**.

Each output is just the running digit product tested with modulo — a remainder check, nothing fancier.

Intuition

Picture a bouncer at a club door, counting people in one at a time until the headcount hits exactly the number the fire code allows. That's the exact shape of this problem: scan forward, one candidate at a time, until a condition clicks true. It's the same underlying pattern used to generate the next valid license plate, or to throttle a counter until it satisfies some rule.

The naive instinct is to worry: what if `t` is large and unlucky, and we have to scan for a very long time before finding a digit product that cooperates? This is where the real insight of the problem lives.

Look at any ten consecutive integers — say 21 through 30. Exactly one of them ends in zero. The moment a number ends in `0`, its digit product is `0`, because zero times anything is zero. And zero is divisible by every integer from 1 to 10 (which covers the constraint range for `t` in this problem). Call this the **Zero Safety Net**.

That means no matter how unlucky you get with `n` and `t`, you are guaranteed to find a valid answer within the next 9 numbers, because the 10th number in any consecutive run has a trailing zero. This single observation converts a search that looks unbounded into one with a hard, small ceiling.

Once you see the Zero Safety Net, the algorithm writes itself: start at `n`, check the digit product, and if it fails, move to `n + 1`. Repeat. You will never need more than ten checks.

Brute Force Solution

Idea: Starting at `n`, compute the digit product of each candidate and test it against `t` using modulo. Increment and repeat until a candidate passes.

Algorithm: 1. Set `candidate = n`. 2. Compute the digit product of `candidate`. 3. If `digit_product % t == 0`, return `candidate`. 4. Otherwise, increment `candidate` and go to step 2.

Advantages: - Extremely simple to write and reason about. - No edge-case handling needed beyond the loop itself.

Disadvantages: - At first glance it looks unbounded, which can make it feel risky to write under interview pressure without proving a bound.

Time complexity: O(1) — bounded by the Zero Safety Net (see below), each check touches at most a couple of digits. **Space complexity:** O(1) — no auxiliary data structures.

This "brute force" is actually the optimal solution here, because the Zero Safety Net caps it. There isn't a smarter algorithm to reach for — recognizing the bound *is* the optimization.

Optimal Solution

The optimal solution is the same loop as above, but understanding *why* it's optimal is the point.

Step	What happens
1	Start at `n`.
2	Convert the number to its digits and multiply them into a running product (starting at 1).
3	Check `product % t == 0`.
4	If true, return the candidate. If false, move to the next integer.
5	Guaranteed to terminate within 9 increments thanks to a trailing-zero candidate.

There's no need to precompute factorizations of `t` or reason about which digits divide `t` — the trailing-zero guarantee makes all of that unnecessary for this version of the problem. (A follow-up variant of this problem removes `n` entirely and asks you to *construct* the smallest number from scratch — that one genuinely needs digit factorization, but this one doesn't.)

Python Code

```
def smallest_number(n: int, t: int) -> int:
    candidate = n
    while True:
        product = 1
        for digit_char in str(candidate):
            product *= int(digit_char)
        if product % t == 0:
            return candidate
        candidate += 1
```

Code Walkthrough

- `candidate = n`: we start the search exactly at `n`, since `n` itself may already be a valid answer.
- `while True`: the Zero Safety Net guarantees this loop exits within 10 iterations, so an unconditional loop is safe here.

- `product = 1`: initialized to 1, not 0, because we're multiplying. Starting at 0 would make every product 0 and every check pass — a classic bug.
- `for digit_char in str(candidate)`: converting the number to a string lets us iterate over its digits one character at a time.
- `product *= int(digit_char)`: each digit character is converted back to an integer and folded into the running product.
- `if product % t == 0`: the modulo operator checks the remainder. A remainder of zero means the product divides evenly by `t`.
- `candidate += 1`: if the check fails, move to the next integer and try again.

Dry Run

Let's trace `n = 29`, `t = 5`.

- `candidate = 29`: digits are `2` and `9`. Product = `2 × 9 = 18`. `18 % 5 = 3` — not zero. Move on.
- `candidate = 30`: digits are `3` and `0`. Product = `3 × 0 = 0`. `0 % 5 = 0` — success.

Return **30**. Notice the trailing zero in 30 is what guaranteed a hit — this is the Zero Safety Net in action.

Complexity Analysis

Time complexity: O(1). The loop runs at most 10 times because of the Zero Safety Net, and each iteration only processes the digits of a number bounded by the input constraints — a fixed, small amount of work. Since neither the iteration count nor the per-iteration work grows with the input size, the whole operation is constant time.

Space complexity: O(1). We only track a running product and a candidate integer — no arrays, no recursion stack, no auxiliary storage that scales with input.

Alternative Solutions

You could precompute the digit product incrementally as the last digit rolls over (since only the last digit changes as you increment by 1, except at multiples of ten). This would save recomputing the entire product each time. In practice, this isn't worth doing here — the loop already runs at most 10 times, so the "optimization" saves a trivial amount of work while adding real complexity and bug

surface. This is a case where the simpler brute-force loop is the correct engineering choice, not just the easy one.

Edge Cases

- `n` **already satisfies the condition:** e.g., `n = 10`, `t = 2` — digit product is `0`, immediately divisible. Must check `n` itself, not `n + 1`.
- `t = 1`**:** any digit product is divisible by 1, so the answer is always `n` itself.
- `n` **contains a zero digit already:** the digit product is `0`, which is divisible by any `t`, so the answer is `n`.
- **Large** `t` **relative to digit range:** doesn't matter — the Zero Safety Net still caps the search at 9 steps past `n`.
- **Single-digit** `n`**:** the loop still works correctly since `str(n)` still has one character to iterate over.

Common Mistakes

1. **Initializing** `product` **to 0 instead of 1.** This is the big one — it makes every digit product read as 0, so every candidate looks valid immediately, and you can't tell a real answer from broken logic.
2. **Initializing** `product` **outside the while loop.** If you don't reset it to 1 at the start of each candidate check, the product keeps compounding across attempts instead of representing a single number's digits.
3. **Starting the search at** `n + 1` **instead of** `n`**.** This skips the case where `n` is already the answer.
4. **Forgetting to convert digit characters back to integers.** Multiplying string characters directly will throw a type error or produce incorrect results.
5. **Assuming the search could run indefinitely and adding unnecessary bounds or fallback logic.** The Zero Safety Net already guarantees termination within 10 steps — extra safeguards are just noise.

Interview Questions

- "How do you know this loop terminates quickly? Can you prove a bound?" (This is the core question the problem is testing.)
- "What's the digit product of a number containing a zero digit, and why does that matter here?"

- "How would this change if `t` could be much larger, say up to 100?" (The Zero Safety Net argument would need re-examination since not every `t` up to 100 divides 0's factors the same way — actually zero is divisible by *any* integer, so the bound still holds regardless of `t`'s size.)
- "Can you solve the reverse problem — construct the smallest number with a digit product divisible by `t`, without being given a starting `n`?" (This is LeetCode's follow-up variant, which needs digit factorization instead of scanning.)

Similar Problems

- **LeetCode 3346 (Smallest Divisible Digit Product II):** the sequel — you must construct the smallest valid number from scratch instead of scanning from `n`, using factorization of `t` into single digits.
- **LeetCode 1291 (Sequential Digits):** another digit-manipulation problem that rewards converting numbers to strings and iterating characters.
- **LeetCode 400 (Nth Digit):** builds the same muscle of reasoning about digits positionally rather than as a single integer.
- **LeetCode 1969 (Minimum Non-Zero Product of the Array Elements):** a different flavor of "product of digits/values" reasoning under a divisibility-like constraint.

Key Takeaways

The core lesson here isn't the code — it's the reasoning that justifies the code. Brute force looks scary until you can bound it. The Zero Safety Net — the guarantee that a trailing zero appears within any run of ten consecutive integers — turns an apparently unbounded scan into a provably constant-time algorithm. Whenever you notice a "reset" value guaranteed to appear nearby in your search space, stop worrying about worst-case scenarios and simulate forward with confidence.

Watch the Video

Prefer to see this solved live, including a full dry run and a quick quiz to test your understanding? Watch the complete walkthrough here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series dedicated to taking LeetCode problems apart until they stop being scary. Each episode picks a problem, strips away the intimidating

framing, and rebuilds the solution from first principles — with an emphasis on the intuition an experienced engineer uses to find the answer, not just the final code.

Call To Action

If this walkthrough helped the problem click, consider liking the video on YouTube and subscribing to the channel — it's a small channel, and every subscribe genuinely helps it grow. For deeper dives and written companions to every video, subscribe to the Substack newsletter. And drop a comment with your answer to the quick quiz, or let us know what problem you'd like broken down next.

Quick Review

Pattern: search min N with digit-based check over tiny fixed range?

Brute-force bounded search — trigger: answer is guaranteed within a small constant range (1-9), so just iterate and test.

Time complexity of checking candidates 1 through 9?

O(1) — at most 9 candidates checked, each check does constant digit-product work.

Space complexity of this approach?

O(1) — only a few integer variables (candidate, product, digits), no extra structures.

Trickiest edge case / common bug here?

Digit '0' makes the product 0, which is divisible by anything — must special-case or it breaks the 'divisible' logic.

Bounded linear scan vs binary search — when to pick which?

Bounded scan when the valid range is tiny (≤10); binary search when range is large but answer is monotonic.

Interview follow-up: why not just increment n and check divisibility directly?

Works but scans is unbounded upward; framing it as checking digit products of 1-9 first shows you spotted the bound.

Smallest Divisible Digit Product I — free Python cheat sheet